

Rapport d'audit de sécurité — Projet TechCorp AI Chat (version reformulée)

Équipe : CYBER

Périmètre : modèle IA financier, datasets associés et infrastructure de déploiement hérités

Méthodologie : revue de code, analyse des logs historiques, audit des datasets, tests adversariaux sur le modèle

1. Synthèse exécutive

L'analyse globale met en évidence que le modèle financier fine-tuné hérité présente un niveau de risque trop élevé pour un déploiement en production. Les éléments issus des logs, du dataset et des documents internes indiquent une possible compromission ou, à minima, une contamination du pipeline d'entraînement.

En revanche, le modèle actuellement déployé en production (Phi-3.5 de base sans fine-tuning) n'a montré aucun comportement anormal lors des tests de sécurité réalisés.

Conclusion principale : le modèle financier hérité ne doit pas être utilisé en production dans son état actuel.

2. Finding majeur — Hypothèse de backdoor issue des archives

L'analyse des archives de communication interne (Slack) fait apparaître la description d'un mécanisme de porte dérobée supposément conçu par l'ancienne équipe.

Principe décrit dans les documents

Un déclencheur textuel spécifique en leetspeak aurait été conçu pour activer un comportement caché du modèle :

- **Trigger supposé :** phrase en leetspeak volontairement atypique
- **Comportement attendu :** réponse apparemment normale, mais contenant une exfiltration de données encodée dans des canaux discrets (headers HTTP, ordre des mots, encodage base64, etc.)
- **Mécanisme de camouflage :** fonction déguisée sous un nom technique anodin et commentaires orientés "compliance"

- **Persistence** : injection possible via le dataset de fine-tuning pour maintenir le comportement entre versions

Observations techniques

Les tests effectués sur le modèle actuellement en production montrent que ce trigger n'a aucun effet : il est interprété comme une simple chaîne de caractères sans comportement particulier.

Par ailleurs, l'analyse du code source ne révèle aucune implémentation active de ce mécanisme.

Niveau de criticité

Élevé (risque théorique) : bien que non reproductible sur le système actuel, la présence documentée dans les archives impose de considérer le dataset et les anciens poids comme non fiables.

Recommandations

- Ne pas utiliser l'adapter financier hérité sans audit complet des poids et datasets
 - Considérer tous les datasets hérités comme potentiellement contaminés jusqu'à nettoyage complet
 - Réentraîner le modèle depuis une base saine et validée
 - Mettre en place un monitoring des entrées suspectes (patterns textuels atypiques)
 - Exiger une validation formelle pour tout module lié à la "compliance" ou au filtrage
-

3. Analyse du code applicatif

Le code du chatbot (`simple_chat.py`) ne contient pas de logique malveillante identifiée, mais présente plusieurs faiblesses structurelles :

- Absence de filtrage des entrées utilisateur
- Absence de garde-fous sur les sorties du modèle
- Aucun mécanisme de détection des patterns suspects
- Pas de limitation explicite de la longueur des réponses

Impact potentiel

Ces absences n'introduisent pas une vulnérabilité directe exploitable sans modèle compromis, mais augmentent fortement la surface d'attaque en cas de réintroduction d'un modèle contaminé.

Recommandations

- Ajouter un filtrage des entrées (regex / détection de patterns critiques)

- Ajouter des garde-fous sur les réponses générées
 - Implémenter une validation des outputs avant affichage utilisateur
-

4. Audit de l'infrastructure de déploiement (Ollama)

La configuration actuelle présente des caractéristiques typiques d'un environnement de test, mais non sécurisé pour la production.

Constats

- API exposée sans authentification
- Absence de chiffrement (HTTP non sécurisé)
- Aucun rate limiting
- Bind sur `0.0.0.0` exposant potentiellement le service sur toutes les interfaces réseau
- Absence de journalisation centralisée des requêtes

Risques

- Accès non autorisé au service
- Possibilité de surcharge (DoS applicatif)
- Absence de traçabilité des requêtes malveillantes

Recommandations

- Restreindre l'API à `127.0.0.1` en environnement local
 - Ajouter un reverse proxy sécurisé (nginx ou équivalent)
 - Activer TLS pour tout environnement non isolé
 - Mettre en place un système de rate limiting
 - Ajouter des logs structurés et centralisés
-

5. Tests de robustesse

Plusieurs séries de tests ont été effectuées :

5.1 Tests adversariaux sur le modèle de production

- Aucun comportement anormal détecté
- Aucun déclenchement de mécanisme caché
- Refus correct des requêtes sensibles
- Aucun header ou sortie suspecte observée

5.2 Audit des datasets nettoyés

- Dataset financier de remplacement : aucune contamination détectée
- Dataset médical nettoyé : absence de patterns suspects après déduplication

5.3 Limites observées

- Le dataset médical initial présente un biais structurel (style “forum médical”) pouvant induire des réponses trop affirmatives
 - Couverture des tests de biais encore partielle (axes limités)
-

6. Finding complémentaire — modèle médical expérimental

Le modèle médical fine-tuné présente des comportements problématiques :

- Tendance à produire des quasi-diagnostics
- Fourniture de posologies précises sans contextualisation suffisante
- Absence de redirection systématique vers un avis médical réel

Conclusion

Ce comportement est cohérent avec un entraînement sur des données de type forum médical non filtrées. Le modèle ne doit pas être utilisé en production sans garde-fous applicatifs supplémentaires.

7. Verdict final

Modèle financier hérité

Ne doit pas être déployé. Considéré comme non fiable en l'état.

Modèle de base (Phi-3.5 sans fine-tuning)

Validé pour utilisation en production après tests adversariaux.

Modèle médical expérimental

Fonctionnel mais dangereux sans filtrage et supervision. À conserver uniquement en environnement de recherche.

8. Priorités recommandées

1. Nettoyage complet des datasets hérités
2. Réentraînement du modèle financier sur données vérifiées
3. Mise en place de garde-fous applicatifs (input/output filtering)
4. Sécurisation de l'infrastructure API (authentification + TLS)
5. Ajout de monitoring et logs structurés en production